Code lab 1 - Create a Counter App

In this code lab, you will learn how to create a simple counter app in flutter. This app will help you to understand the basic concepts of flutter.

Step 1: Create Flutter Project

Using Android Studio or Visual Code

Step 2: Define the Main Entry Point

Open main.dart file and define the main entry point of the app.

```
import 'package:flutter/material.dart';
void main() {
  runApp(const CounterApp());


}
```

import

First, we have the import statement. This imports the material.dart package which helps in Implement Material Design throughout the app, and for most of the Dart files used for UI design, you have to import this package.

Main

The `main()` and `runApp()` function definitions are the same as in the default app. The `runApp()` function takes as its argument a Widget, which the Flutter framework expands and displays to the screen of the app at run time.
You have the main function, which has the ***runApp*** method containing the first Widget as its argument; in this case, it is called ***CounterApp.***

Step 3: Create the CounterApp Widget

Now, you need to create the CounterApp widget.

```
class CounterApp extends StatelessWidget {
  const CounterApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Counter App',
```

```
      home: CounterScreen(),
    );
  }
}
```

CounterApp is the **root widget**, as it's the widget that's passed into runApp. Within this widget, there's a build method that returns another widget called MaterialApp. Essentially, this is what a Flutter app is: a composition of widgets that make up a tree structure called the **widget tree.**

Your job as a Flutter developer is to compose widgets from the SDK into larger, custom widgets that display a UI.

Stateless Widget (CounterApp)

This widget is the root of the app. Here, inside the build method, it returns a ***Material App*** widget containing the three properties: title, theme & home.

title
A one-line description used by the device to identify the app for the user. On **Android**, the titles appear above the task manager's app snapshots which are displayed when the user presses the "recent apps" button. On **iOS**, this value cannot be used.

theme
This property defines the ***ThemeData*** widget containing information about different colors, fonts and other theme data that would be used throughout the app. Here, only one property is defined, ***primarySwatch*** which stores the color blue. But, in your app, you can define any number of theme properties you want.

home
This property contains a Stateful Widget, ***CounterScreen*** . When you start this app for the first time this is the widget that will be displayed as the first screen on your device.

Step 4: Create the CounterScreen Widget

Now, create the CounterScreen widget.

```
class CounterScreen extends StatefulWidget {
  @override
  _CounterScreenState createState() => _CounterScreenState();
}
```

We have the overridden createState method which returns the instance of the class **CounterScreenState.**

State class

In this class at first, a private integer variable ***_counter*** is initialized to zero. After that, we have a private void method defined called ***incrementCounter*** and ***decrementCounter.***

```
class _CounterScreenState extends State<CounterScreen> {
  // Define the counter variable
  int _counter = 0;

  // Define the increment and decrement counter methods
  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  void _decrementCounter() {
    setState(() {
      _counter--;
    });
  }
```

This method calls the setState method, this setState is used to rebuild the widget tree. In this counter app, we have incremented the ***_counter*** variable within this setState. If we just increment the ***_counter*** variable without defining any setState method, then this variable would get updated behind the scene but we will not see any UI changes as the widget tree would not be rebuilt

`Build method`

Now, we have the overridden **build** method. This method is rerun every time setState is called, for instance as done by the ***_incrementCounter*** method above.

The Flutter framework has been optimized to make rerunning build methods fast, so that you can just rebuild anything that needs updating rather than having to individually change instances of widgets.

In this app, the build method returns a Scaffold widget containing three properties: Scaffold, appBar, body & floatingActionButton

```
.
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter App')),
      body: Center(
```

```
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              '$_counter',
              style: const TextStyle(fontSize: 72),
            ),
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceEvenly,
              children:[
                FloatingActionButton(
                  onPressed: _incrementCounter,
                  tooltip: 'Add',
                  child: const Icon(Icons.add),
                ),
                FloatingActionButton(
                  onPressed: _decrementCounter,
                  tooltip: 'Subtract',
                  child: const Icon(Icons.remove),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Scaffold

Scaffold is another build in app that gives us a standard mobile app layout. You'll most likely use this on every page of your app

appBar

This property takes the ***AppBar*** widget containing a Text widget which displays the title of the app.

the bar accross the top of the app

appBar: AppBar(title: const Text('Counter App')),

body

This property contains a ***Column*** widget which is centered using the ***Center*** widget. The column

widget contains one text widget, to display a text the number of counts, and style to define the text **style** for this text widget.. The row contains two floating action buttons.

```
body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text(
              'You have pushed the button this many times:',
            ),
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.display1,
            ),
          ],
        ),
      ),
```

Column is also layout widget. It takes a List of children and arranges them vertically. By default, it sizes itself to fit its children horizontally, and tries to be as tall as its parent.

Column has various properties to control how it sizes itself and how it positions its children. Here we use mainAxisAlignment to center the children vertically; the main axis here is the vertical axis because Columns are vertical (the cross axis would be horizontal).

mainAxisAlignment and **crossAxisAlignment** should feel very familiar if you're used to using CSS's Flexbox or Grid.

Text takes a String as its first argument. We are passing in the value of the counter as an interpolated String.

`floatingActionButton`

Floating action buttons are special button

It takes the **FloatingActionButton** widget which displays a FAB at the bottom right corner of the screen. The FAB widget takes three properties: onPressed, tooltip & child .

```
FloatingActionButton(
                  onPressed: _incrementCounter,
                  tooltip: 'Add',
                  child: const Icon(Icons.add),
                ),
```

`onPressed`

The onPressed property is used to call the **_incrementCounter** method which increments the counter by 1 and rebuilds the widget tree.

`tooltip`

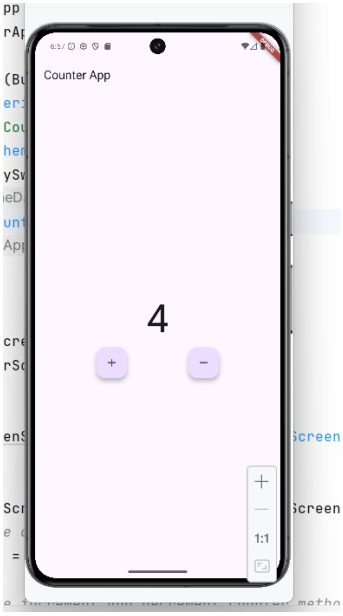
This property can take a **String**. The text is displayed when the user long-presses on the button and is used for **accessibility**, which means that it will not be displayed in normal circumstances.

`child`

As you know, this child widget is used to show anything inside the parent widget. Here, it is used to show a **add** icon.

Step 5: Run the App

Now, you can run the app using Android Studio or Visual Studio code.



`Final Notes`

So in simple words, the working of this app is that, when the + FAB (Floating Action Button) is pressed the _incrementCounter method is called which increments the counter and rebuilds the widget tree.

`Hot Reload`

1. Run the app on the emulator or a real device.
2. Let's change the primarySwatch color to red.

3. Save the file [Command + S (for MAC), or Ctrl + S (for Windows)]. And you would immediately see that the color of the appBar and the FAB change to red.